

Linux Fundamentals — Task 1

Commands — One by One

Each command is presented separately with what it does, how to use it, an example, and an easy idea.

1. Navigation Commands

`pwd`

What it does:

Shows your current working directory.

How to use:

```
pwd
```

Example:

```
pwd
```

Easy idea: `pwd` = Where am I?

`ls`

What it does:

Lists files and directories in the current directory.

How to use:

```
ls
```

Example:

```
ls
```

Easy idea: `ls` = What is here?

`ls -l`

What it does:

Shows a detailed listing, including permissions and ownership.

How to use:

```
ls -l
```

Example:

```
ls -l
```

Easy idea: Detailed listing.

`ls -a`

What it does:

Shows hidden files and directories.

How to use:

```
ls -a
```

Example:

```
ls -a
```

Easy idea: Show hidden files too.

`ls -la`

What it does:

Shows a detailed listing including hidden files.

How to use:

```
ls -la
```

Example:

```
ls -la
```

Easy idea: Detailed + hidden files.

`ls -lh`

What it does:

Shows a detailed listing with human-readable file sizes.

How to use:

```
ls -lh
```

Example:

```
ls -lh
```

Easy idea: Detailed list with readable sizes.

`cd`

What it does:

Moves into a directory.

How to use:

```
cd
```

Example:

```
cd Documents
```

Easy idea: Move to another directory.

`cd ..`

What it does:

Moves one directory up to the parent directory.

How to use:

```
cd ..
```

Example:

```
cd ..
```

Easy idea: Go one directory up.

`cd ~`

What it does:

Moves to your home directory.

How to use:

```
cd ~
```

Example:

```
cd ~
```

Easy idea: Go home.

`cd /`

What it does:

Moves to the filesystem root directory.

How to use:

```
cd /
```

Example:

```
cd /
```

Easy idea: Go to the top of the filesystem.

2. User & Group Commands

`id`

What it does:

Shows your UID, GID, and group memberships.

How to use:

```
id
```

Example:

```
id
```

Easy idea: Show user and group identity.

`cat /etc/passwd`

What it does:

Displays local user-account information.

How to use:

```
cat /etc/passwd
```

Example:

```
cat /etc/passwd
```

Easy idea: View user-account information.

`cat /etc/group`

What it does:

Displays local group information.

How to use:

```
cat /etc/group
```

Example:

```
cat /etc/group
```

Easy idea: View group information.

`sudo chown`

What it does:

Changes the owner of a file. `sudo` may be required.

How to use:

```
sudo chown
```

Example:

```
sudo chown arjun report.txt
```

Easy idea: `chown` = Change owner.

`sudo chown :`

What it does:

Changes both the owner and group.

How to use:

```
sudo chown :
```

Example:

```
sudo chown arjun:security report.txt
```

Easy idea: Change owner + group.

`sudo chgrp`

What it does:

Changes the group associated with a file.

How to use:

```
sudo chgrp
```

Example:

```
sudo chgrp security report.txt
```

Easy idea: `chgrp` = Change group.

3. Permission Commands

`chmod`

What it does:

Changes file or directory permissions.

How to use:

```
chmod
```

Example:

```
chmod 755 script.sh
```

Easy idea: `chmod` = Change permissions.

`chmod 600`

What it does:

Owner can read/write; group and others have no permissions.

How to use:

```
chmod 600
```

Example:

```
chmod 600 secret.txt
```

Easy idea: `600` = Owner read/write only.

`chmod 755`

What it does:

Owner has rwx; group and others have r-x.

How to use:

```
chmod 755
```

Example:

```
chmod 755 script.sh
```

Easy idea: `755` = Owner all; group/others read + execute.

4. Process Commands

`ps`

What it does:

Shows processes associated with the current terminal/session.

How to use:

```
ps
```

Example:

```
ps
```

Easy idea: View processes.

`ps aux`

What it does:

Shows a broader list of running processes.

How to use:

```
ps aux
```

Example:

```
ps aux
```

Easy idea: View many running processes.

`top`

What it does:

Provides a live view of processes and resource usage. Press `q` to exit.

How to use:

```
top
```

Example:

```
top
```

Easy idea: Live process monitoring.

`htop`

What it does:

Provides an interactive process viewer when installed.

How to use:

```
htop
```

Example:

```
htop
```

Easy idea: Interactive process monitoring.

`echo \$\$`

What it does:

Shows the PID of the current shell.

How to use:

```
echo $$
```

Example:

```
echo $$
```

Easy idea: Show current shell PID.

`kill`**What it does:**

Sends the default termination signal to a process.

How to use:

```
kill
```

Example:

```
kill 1234
```

Easy idea: Send a signal to a process.

`kill -15`**What it does:**

Sends SIGTERM and requests graceful termination.

How to use:

```
kill -15
```

Example:

```
kill -15 1234
```

Easy idea: Request graceful termination.

`kill -9`**What it does:**

Sends SIGKILL and forcibly terminates a process; use carefully.

How to use:

```
kill -9
```

Example:

```
kill -9 1234
```

Easy idea: Force termination.

`kill -STOP`**What it does:**

Stops/suspends a process.

How to use:

```
kill -STOP
```

Example:

```
kill -STOP 1234
```

Easy idea: Suspend the process.

`kill -CONT`**What it does:**

Continues a stopped process.

How to use:

```
kill -CONT
```

Example:

```
kill -CONT 1234
```

Easy idea: Continue the process.

`jobs`

What it does:

Shows background jobs managed by the current shell.

How to use:

```
jobs
```

Example:

```
jobs
```

Easy idea: Show shell jobs.

`fg`

What it does:

Brings a shell job to the foreground.

How to use:

```
fg
```

Example:

```
fg
```

Easy idea: Bring a job to foreground.

`bg`

What it does:

Continues a stopped shell job in the background.

How to use:

```
bg
```

Example:

```
bg
```

Easy idea: Continue a job in background.

5. System Service Commands

`systemctl status`

What it does:

Shows the current status of a systemd service.

How to use:

```
systemctl status
```

Example:

```
systemctl status ssh
```

Easy idea: Check the service.

`sudo systemctl start`

What it does:

Starts a service now.

How to use:

```
sudo systemctl start
```

Example:

```
sudo systemctl start ssh
```

Easy idea: Start now.

`sudo systemctl stop`

What it does:

Stops a service now.

How to use:

```
sudo systemctl stop
```

Example:

```
sudo systemctl stop ssh
```

Easy idea: Stop now.

`sudo systemctl restart`

What it does:

Stops and starts a service again.

How to use:

```
sudo systemctl restart
```

Example:

```
sudo systemctl restart ssh
```

Easy idea: Stop + start again.

`sudo systemctl enable`

What it does:

Configures a service to start automatically at boot.

How to use:

```
sudo systemctl enable
```

Example:

```
sudo systemctl enable ssh
```

Easy idea: Start automatically at boot.

`sudo systemctl disable`

What it does:

Disables automatic startup at boot.

How to use:

```
sudo systemctl disable
```

Example:

```
sudo systemctl disable ssh
```

Easy idea: Don't start automatically at boot.

`systemctl is-active`

What it does:

Checks whether a service is currently active.

How to use:

```
systemctl is-active
```

Example:

```
systemctl is-active ssh
```

Easy idea: Check if it is active.

`sudo systemctl enable --now`

What it does:

Enables the service at boot and starts it immediately.

How to use:

```
sudo systemctl enable --now
```

Example:

```
sudo systemctl enable --now ssh
```

Easy idea: Enable at boot + start now.

6. Permission Numbers

`r`

What it does:

Read permission.

How to use:

```
r = 4
```

Example:

```
4
```

Easy idea: Read = 4.

`w`

What it does:

Write permission.

How to use:

```
w = 2
```

Example:

```
2
```

Easy idea: Write = 2.

`x`

What it does:

Execute permission.

How to use:

```
x = 1
```

Example:

```
1
```

Easy idea: Execute = 1.

`rwx`

What it does:

Read + write + execute.

How to use:

```
rwX = 4 + 2 + 1
```

Example:

```
7
```

Easy idea: rwx = 7.

`r-x`

What it does:

Read + execute.

How to use:

```
r-x = 4 + 0 + 1
```

Example:

```
5
```

Easy idea: r-x = 5.

`r--`

What it does:

Read only.

How to use:

```
r-- = 4 + 0 + 0
```

Example:

```
4
```

Easy idea: r-- = 4.

`---`

What it does:

No permissions.

How to use:

```
--- = 0 + 0 + 0
```

Example:

```
0
```

Easy idea: No access = 0.

7. Important Linux Paths

`/`

What it does:

The root of the filesystem.

How to use:

```
cd /
```

Example:

```
cd /
```

Easy idea: Top of the filesystem.

`/home`

What it does:

Contains normal users' home directories.

How to use:

```
ls /home
```

Example:

```
ls /home
```

Easy idea: Normal user homes.

`/root`

What it does:

The root user's home directory.

How to use:

```
ls /root
```

Example:

```
ls /root
```

Easy idea: Root user's home.

`/etc`

What it does:

Contains configuration files.

How to use:

```
ls /etc
```

Example:

```
ls /etc
```

Easy idea: Configuration.

`/var/log`

What it does:

Contains system and application logs.

How to use:

```
ls /var/log
```

Example:

```
ls /var/log
```

Easy idea: Logs.

`/tmp`

What it does:

Contains temporary files.

How to use:

```
ls /tmp
```

Example:

```
ls /tmp
```

Easy idea: Temporary files.

`/proc`

What it does:

Provides process and kernel/system information.

How to use:

```
ls /proc
```

Example:

```
ls /proc
```

Easy idea: Process/system information.

`/dev`

What it does:

Contains device interfaces.

How to use:

```
ls /dev
```

Example:

```
ls /dev
```

Easy idea: Device interfaces.

Task 1 complete: Navigation, users and groups, permissions, processes, system services, permission numbers, and important Linux paths.